Objetos y tipos de datos en Python

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

En Python, todo es un objeto y su comportamiento está determinado por su tipo y mutabilidad. Este artículo explora los conceptos fundamentales de objetos en Python, con énfasis en la diferencia entre asignación por referencia y por valor, los principales tipos de datos built-in y el manejo práctico de las colecciones más utilizadas; listas (mutables y ordenadas), tuplas (inmutables y eficientes en memoria) y diccionarios (mutables, no ordenados y basados en claves-valor). Se presentan ejemplos claros sobre indexación, métodos principales, evaluación de membresía, copia superficial y profunda, así como consideraciones importantes de consumo de memoria y buenas prácticas al trabajar con estas estructuras.

Palabras Claves: Objetos Python, Mutabilidad, Referencias vs copia

Tabla de contenidos

Introduction **4**

1 Listas **4**
1.1 Creación de listas 4
1.2 Listas heterogéneas 4
1.3 Indexación de listas 5
1.4 Slicing de listas 5
1.5 Modificación de elementos 6
1.6 Listas anidadas 6

2 Asignación por referencia vs por valor **7**
2.1 Asignación por referencia 7
2.2 Copia superficial (shallow copy) 8
2.3 Problema con listas anidadas 8

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

2.4 Copia profunda (deep copy) . . . 9
2.5 Ejemplo aplicado: Escenarios económicos . . . 9

3 Operaciones con listas 10
3.1 Concatenación y repetición . . . 10
3.2 Pertenencia a listas . . . 11
3.3 Comparación de listas . . . 11
3.4 Eliminar elementos . . . 11

4 Métodos de listas 12
4.1 Funciones básicas . . . 12
4.2 Método append() . . . 13
4.3 Método extend() . . . 13
4.4 Método insert() . . . 14
4.5 Método remove() . . . 14
4.6 Método pop() . . . 14
4.7 Método index() . . . 15
4.8 Método count() . . . 15
4.9 Método sort() . . . 15
4.10 Función sorted() . . . 16
4.11 Método reverse() . . . 16

5 Diccionarios 17
5.1 Creación de diccionarios . . . 17
5.2 Acceso a elementos . . . 17
5.3 Modificación de diccionarios . . . 18
5.4 Verificación de claves . . . 19

6 Operaciones con diccionarios 19
6.1 Métodos keys(), values(), items() . . . 19
6.2 Método update() . . . 20
6.3 Método pop() . . . 20
6.4 Método copy() . . . 21
6.5 Método clear() . . . 21
6.6 Diccionarios anidados . . . 22

7 Tuplas 22
7.1 Creación de tuplas . . . 22
7.2 Características de tuplas . . . 23
7.3 Ventajas de tuplas . . . 24
7.4 Desempaquetado de tuplas . . . 24
7.5 Métodos de tuplas . . . 25

8 Asignación múltiple con diccionarios 25
8.1 Retorno múltiple de funciones . . . 26

9 Aplicaciones económicas **26**
9.1 Ejemplo 1: Análisis de cartera de inversiones 26
9.2 Ejemplo 2: Análisis de comercio exterior 27
9.3 Ejemplo 3: Sistema de calificación de riesgo 28

10 Ejercicios prácticos **30**
10.1 Ejercicio 1: Administrador de presupuesto familiar 30
10.2 Ejercicio 2: Análisis de series de tiempo económicas 32
10.3 Ejercicio 3: Sistema de inventario 33

11 Conclusión **34**

12 Próximos pasos **34**

13 Recursos adicionales **35**

14 Publicaciones Similares **35**

Objetos y tipos de datos en Python

En esta tercera guía exploraremos las estructuras de datos fundamentales de Python: listas, diccionarios y tuplas. Estas estructuras son esenciales para organizar y manipular datos económicos de manera eficiente.

1 Listas

Las listas son colecciones ordenadas y mutables de elementos en Python. Son uno de los tipos de datos más versátiles y utilizados, especialmente en análisis económico donde necesitamos trabajar con series de datos.

1.1 Creación de listas

```
# Lista de precios
precios = [10.50, 12.30, 15.80, 9.90, 11.20]
print(precios)

# Lista de sectores económicos
sectores = ['Agricultura', 'Minería', 'Manufactura', 'Servicios']
print(sectores)

# Lista de años
anos = [2019, 2020, 2021, 2022, 2023]
print(anos)

# Crear lista con la función list()
tasas_interes = list([0.05, 0.06, 0.055, 0.065, 0.07])
print(tasas_interes)

# Lista vacía
datos_vacios = []
print(datos_vacios)
```

1.2 Listas heterogéneas

Las listas pueden contener elementos de diferentes tipos:

```
# Lista mixta con datos económicos
datos_empresa = ['TechCorp', 2015, 1500000.50, True, ['Lima', 'Arequipa']]
print(datos_empresa)
# ['TechCorp', 2015, 1500000.50, True, ['Lima', 'Arequipa']]

# Estructura de datos compleja
indicador_economico = [
    'PIB',
    242632,  # Millones de USD
    2023,
    [2.4, 4.0, -11.0, 13.3, 2.7],  # Tasas de crecimiento histórico
```

```
    {'moneda': 'USD', 'unidad': 'millones'}
]
```

1.3 Indexación de listas

Las listas en Python usan indexación de base cero (el primer elemento está en la posición 0):

```
# Datos de inflación mensual
inflacion_mensual = [0.5, 0.3, 0.4, 0.6, 0.2, 0.5, 0.7, 0.4, 0.5, 0.6, 0.3, 0.4]

# Acceder a elementos individuales
enero = inflacion_mensual[0]
print(f"Inflación de enero: {enero}%")  # 0.5%

febrero = inflacion_mensual[1]
print(f"Inflación de febrero: {febrero}%")  # 0.3%

diciembre = inflacion_mensual[11]
print(f"Inflación de diciembre: {diciembre}%")  # 0.4%

# Indexación negativa (desde el final)
ultimo_mes = inflacion_mensual[-1]
print(f"Último mes: {ultimo_mes}%")  # 0.4%

penultimo_mes = inflacion_mensual[-2]
print(f"Penúltimo mes: {penultimo_mes}%")  # 0.3%
```

1.4 Slicing de listas

El slicing permite extraer sublistas:

```
# PIB trimestral
pib_trimestral = [50000, 52000, 51500, 53000, 54000, 55500, 56000, 57200]

# Sintaxis: lista[inicio:fin:paso]
# El índice 'fin' no se incluye

# Primer trimestre del primer año
q1_ano1 = pib_trimestral[0]
print(f"Q1 Año 1: {q1_ano1}")

# Todos los trimestres del primer año
ano1 = pib_trimestral[0:4]
print(f"Año 1: {ano1}")  # [50000, 52000, 51500, 53000]

# Todos los trimestres del segundo año
ano2 = pib_trimestral[4:8]
```

```
print(f"Año 2: {ano2}")  # [54000, 55500, 56000, 57200]

# Primeros tres trimestres
primeros_tres = pib_trimestral[:3]
print(f"Primeros 3 trimestres: {primeros_tres}")

# Desde el cuarto trimestre hasta el final
desde_cuarto = pib_trimestral[3:]
print(f"Desde Q4: {desde_cuarto}")

# Todos los elementos (copia de la lista)
todos = pib_trimestral[:]
print(f"Todos: {todos}")

# Con paso: cada 2 elementos
trimestres_pares = pib_trimestral[::2]
print(f"Trimestres impares (Q1, Q3): {trimestres_pares}")

# Invertir lista
invertido = pib_trimestral[::-1]
print(f"Invertido: {invertido}")
```

1.5 Modificación de elementos

Las listas son mutables, podemos cambiar sus elementos:

```
# Proyecciones de crecimiento
crecimiento = [2.5, 3.0, 3.2, 2.8, 3.5]
print(f"Original: {crecimiento}")

# Modificar un elemento
crecimiento[2] = 3.3  # Actualizar proyección del tercer año
print(f"Modificado: {crecimiento}")

# Modificar múltiples elementos con slicing
crecimiento[3:5] = [2.9, 3.6]  # Actualizar últimos dos años
print(f"Actualizado: {crecimiento}")

# Modificar usando indexación negativa
crecimiento[-1] = 3.7  # Cambiar última proyección
print(f"Final: {crecimiento}")
```

1.6 Listas anidadas

Las listas pueden contener otras listas, útil para representar matrices o datos multidimensionales:

```python
# Matriz de datos económicos por región y año
# Filas: regiones, Columnas: años
pib_regional = [
    [100000, 105000, 110000],  # Lima
    [50000, 52000, 54500],     # Arequipa
    [30000, 31500, 33000],     # Cusco
    [45000, 47000, 49000]      # La Libertad
]

# Acceder a PIB de Lima en el segundo año
lima_ano2 = pib_regional[0][1]
print(f"PIB Lima año 2: {lima_ano2}")  # 105000

# Acceder a PIB de Cusco en el tercer año
cusco_ano3 = pib_regional[2][2]
print(f"PIB Cusco año 3: {cusco_ano3}")  # 33000

# Acceder a todos los datos de Arequipa
arequipa_todos = pib_regional[1]
print(f"PIB Arequipa todos los años: {arequipa_todos}")

# Modificar un valor específico
pib_regional[1][2] = 55000  # Actualizar Arequipa año 3
print(f"PIB Arequipa actualizado: {pib_regional[1]}")
```

2 Asignación por referencia vs por valor

Este es uno de los conceptos más importantes y frecuentemente malentendidos en Python. Entender la diferencia puede evitar errores difíciles de detectar.

2.1 Asignación por referencia

Cuando asignamos una lista a otra variable, Python no crea una copia nueva. Ambas variables apuntan al mismo objeto en memoria:

```python
# Lista original de precios
precios_2023 = [100, 105, 110, 108, 112]
print(f"Precios 2023: {precios_2023}")

# Asignación por referencia
precios_2024 = precios_2023  # ¡No es una copia!

# Modificar precios_2024
precios_2024[0] = 120

# Ambas listas cambian
print(f"Precios 2023: {precios_2023}")  # [120, 105, 110, 108, 112]
print(f"Precios 2024: {precios_2024}")  # [120, 105, 110, 108, 112]
```

```
# Verificar que son el mismo objeto
print(f"¿Son el mismo objeto? {precios_2023 is precios_2024}")  # True
```

Diagrama conceptual:

```
precios_2023
                > [120, 105, 110, 108, 112] (un solo objeto en memoria)
precios_2024
```

2.2 Copia superficial (shallow copy)

Para crear una copia independiente de una lista:

```
import copy

# Lista original
inflacion_base = [2.5, 3.0, 2.8, 3.2]
print(f"Inflación base: {inflacion_base}")

# Método 1: usando copy.copy()
inflacion_copia1 = copy.copy(inflacion_base)

# Método 2: usando slicing [:]
inflacion_copia2 = inflacion_base[:]

# Método 3: usando list()
inflacion_copia3 = list(inflacion_base)

# Modificar la copia
inflacion_copia1[0] = 2.7

# Solo cambia la copia
print(f"Inflación base:   {inflacion_base}")   # [2.5, 3.0, 2.8, 3.2]
print(f"Inflación copia1: {inflacion_copia1}") # [2.7, 3.0, 2.8, 3.2]

# Verificar que son objetos diferentes
print(f"¿Son el mismo objeto? {inflacion_base is inflacion_copia1}")  # False
```

2.3 Problema con listas anidadas

La copia superficial no copia las listas internas:

```
import copy

# Lista de listas
datos_economicos = [
    ['Lima', 100000],
    ['Arequipa', 50000],
```

```
    ['Cusco', 30000]
]

# Copia superficial
datos_copia_superficial = copy.copy(datos_economicos)

# Modificar una sublista
datos_copia_superficial[0][1] = 105000

# La lista original también cambia
print(f"Original: {datos_economicos}")
# [['Lima', 105000], ['Arequipa', 50000], ['Cusco', 30000]]

print(f"Copia: {datos_copia_superficial}")
# [['Lima', 105000], ['Arequipa', 50000], ['Cusco', 30000]]
```

2.4 Copia profunda (deep copy)

Para copiar listas que contienen otras listas:

```
import copy

# Lista de listas
datos_economicos = [
    ['Lima', 100000],
    ['Arequipa', 50000],
    ['Cusco', 30000]
]

# Copia profunda
datos_copia_profunda = copy.deepcopy(datos_economicos)

# Modificar la copia
datos_copia_profunda[0][1] = 105000

# Solo cambia la copia profunda
print(f"Original: {datos_economicos}")
# [['Lima', 100000], ['Arequipa', 50000], ['Cusco', 30000]]

print(f"Copia profunda: {datos_copia_profunda}")
# [['Lima', 105000], ['Arequipa', 50000], ['Cusco', 30000]]
```

2.5 Ejemplo aplicado: Escenarios económicos

```
import copy

# Escenario base
```

```
escenario_base = {
    'pib': 242632,
    'inflacion': 3.5,
    'desempleo': 7.2,
    'sectores': ['Minería', 'Agricultura', 'Manufactura']
}

# Crear escenarios alternativos
escenario_optimista = copy.deepcopy(escenario_base)
escenario_pesimista = copy.deepcopy(escenario_base)

# Modificar escenarios
escenario_optimista['pib'] = 250000
escenario_optimista['inflacion'] = 3.0
escenario_optimista['sectores'].append('Tecnología')

escenario_pesimista['pib'] = 235000
escenario_pesimista['inflacion'] = 4.5
escenario_pesimista['sectores'].remove('Manufactura')

# Cada escenario es independiente
print("Escenario Base:")
print(escenario_base)
print("\nEscenario Optimista:")
print(escenario_optimista)
print("\nEscenario Pesimista:")
print(escenario_pesimista)
```

3 Operaciones con listas

3.1 Concatenación y repetición

```
# Datos del primer semestre
primer_semestre = [50000, 52000, 51500]

# Datos del segundo semestre
segundo_semestre = [53000, 54500, 55000]

# Concatenar listas
ano_completo = primer_semestre + segundo_semestre
print(f"Año completo: {ano_completo}")
# [50000, 52000, 51500, 53000, 54500, 55000]

# Repetir listas
patron_base = [100, 110]
patron_repetido = patron_base * 3
print(f"Patrón repetido: {patron_repetido}")
```

```
# [100, 110, 100, 110, 100, 110]

# Aplicación: Crear valores iniciales repetidos
tasas_iniciales = [0.05] * 12  # Tasa fija para 12 meses
print(f"Tasas iniciales: {tasas_iniciales}")
```

3.2 Pertenencia a listas

```
# Sectores económicos
sectores_primarios = ['Agricultura', 'Pesca', 'Minería']

# Verificar pertenencia
print('Minería' in sectores_primarios)        # True
print('Manufactura' in sectores_primarios)    # False
print('Pesca' not in sectores_primarios)      # False
print('Tecnología' not in sectores_primarios) # True

# Aplicación práctica
sector_analizar = 'Agricultura'

if sector_analizar in sectores_primarios:
    print(f"{sector_analizar} es un sector primario")
else:
    print(f"{sector_analizar} no es un sector primario")
```

3.3 Comparación de listas

```
# PIB por trimestre de dos años
pib_2022 = [50000, 52000, 51500, 53000]
pib_2023 = [54000, 55000, 54500, 56000]
pib_2023_copia = [54000, 55000, 54500, 56000]

# Igualdad
print(pib_2022 == pib_2023)        # False
print(pib_2023 == pib_2023_copia)  # True

# Identidad (mismo objeto en memoria)
print(pib_2023 is pib_2023_copia)  # False

# Comparación elemento por elemento
print(pib_2022 < pib_2023)  # True (compara primer elemento diferente)
```

3.4 Eliminar elementos

```python
# Lista de sectores
sectores = ['Agricultura', 'Minería', 'Manufactura', 'Construcción', 'Servicios']
print(f"Original: {sectores}")

# Método 1: del (eliminar por índice)
del sectores[2]  # Elimina 'Manufactura'
print(f"Después de del: {sectores}")

# Método 2: remove() (eliminar por valor)
sectores.remove('Construcción')
print(f"Después de remove: {sectores}")

# Método 3: pop() (eliminar y retornar elemento)
sector_eliminado = sectores.pop(1)  # Elimina y retorna 'Minería'
print(f"Sector eliminado: {sector_eliminado}")
print(f"Lista actual: {sectores}")

# pop() sin argumentos elimina el último elemento
ultimo = sectores.pop()
print(f"Último eliminado: {ultimo}")
print(f"Lista final: {sectores}")

# Método 4: clear() (vaciar toda la lista)
sectores.clear()
print(f"Lista vacía: {sectores}")
```

4 Métodos de listas

4.1 Funciones básicas

```python
# Datos de ventas mensuales (en miles)
ventas = [45, 52, 48, 55, 60, 58, 62, 59, 65, 70, 68, 75]

# Longitud de la lista
num_meses = len(ventas)
print(f"Número de meses: {num_meses}")  # 12

# Valor mínimo
venta_minima = min(ventas)
print(f"Venta mínima: {venta_minima} mil")  # 45

# Valor máximo
venta_maxima = max(ventas)
print(f"Venta máxima: {venta_maxima} mil")  # 75

# Suma total
ventas_totales = sum(ventas)
```

```
print(f"Ventas totales: {ventas_totales} mil")  # 717

# Promedio
ventas_promedio = sum(ventas) / len(ventas)
print(f"Venta promedio: {ventas_promedio:.2f} mil")  # 59.75
```

4.2 Método append()

Agrega un elemento al final de la lista:

```
# Registro de inflación mensual
inflacion = [0.5, 0.3, 0.4]
print(f"Inflación inicial: {inflacion}")

# Agregar nuevos meses
inflacion.append(0.6)
inflacion.append(0.5)
print(f"Inflación actualizada: {inflacion}")

# Agregar una lista como elemento único
inflacion.append([0.7, 0.8])
print(f"Con lista anidada: {inflacion}")
# [0.5, 0.3, 0.4, 0.6, 0.5, [0.7, 0.8]]
```

4.3 Método extend()

Extiende la lista agregando elementos de un iterable:

```
# PIB trimestral
pib_q1_q2 = [50000, 52000]
print(f"PIB Q1-Q2: {pib_q1_q2}")

# Extender con más trimestres
pib_q3_q4 = [51500, 53000]
pib_q1_q2.extend(pib_q3_q4)
print(f"PIB año completo: {pib_q1_q2}")
# [50000, 52000, 51500, 53000]

# Diferencia entre extend y append
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]

lista1.extend([4, 5])
lista2.append([4, 5])

print(f"Con extend: {lista1}")  # [1, 2, 3, 4, 5]
print(f"Con append: {lista2}")  # [1, 2, 3, [4, 5]]
```

4.4 Método insert()

Inserta un elemento en una posición específica:

```
# Tasas de crecimiento
crecimiento = [2.5, 3.0, 3.5, 4.0]
print(f"Original: {crecimiento}")

# Insertar en posición específica
crecimiento.insert(2, 3.2)  # Insertar 3.2 en índice 2
print(f"Después de insert: {crecimiento}")
# [2.5, 3.0, 3.2, 3.5, 4.0]

# Insertar al inicio
crecimiento.insert(0, 2.0)
print(f"Con valor inicial: {crecimiento}")
# [2.0, 2.5, 3.0, 3.2, 3.5, 4.0]
```

4.5 Método remove()

Elimina la primera ocurrencia de un valor:

```
# Sectores para análisis
sectores = ['Minería', 'Agricultura', 'Minería', 'Manufactura']
print(f"Original: {sectores}")

# Eliminar primera ocurrencia de 'Minería'
sectores.remove('Minería')
print(f"Después de remove: {sectores}")
# ['Agricultura', 'Minería', 'Manufactura']

# Intento de eliminar valor inexistente genera error
try:
    sectores.remove('Tecnología')
except ValueError:
    print("Error: 'Tecnología' no está en la lista")
```

4.6 Método pop()

Elimina y retorna un elemento:

```
# Cola de proyectos
proyectos = ['Proyecto A', 'Proyecto B', 'Proyecto C', 'Proyecto D']
print(f"Proyectos: {proyectos}")

# Eliminar y obtener el último proyecto
ultimo_proyecto = proyectos.pop()
print(f"Proyecto completado: {ultimo_proyecto}")
print(f"Proyectos restantes: {proyectos}")
```

```
# Eliminar y obtener un proyecto específico
proyecto_siguiente = proyectos.pop(0)  # Primer proyecto
print(f"Proyecto en ejecución: {proyecto_siguiente}")
print(f"Proyectos pendientes: {proyectos}")
```

4.7 Método index()

Retorna el índice de la primera ocurrencia de un valor:

```
# Sectores económicos
sectores = ['Agricultura', 'Minería', 'Manufactura', 'Servicios', 'Construcción']

# Encontrar índice de un sector
indice_mineria = sectores.index('Minería')
print(f"Índice de Minería: {indice_mineria}")  # 1

indice_servicios = sectores.index('Servicios')
print(f"Índice de Servicios: {indice_servicios}")  # 3

# Búsqueda en rango específico
lista_numeros = [1, 2, 3, 2, 4, 2, 5]
indice = lista_numeros.index(2, 2)  # Buscar 2 desde índice 2
print(f"Índice de 2 desde posición 2: {indice}")  # 3
```

4.8 Método count()

Cuenta las ocurrencias de un valor:

```
# Calificaciones de riesgo
calificaciones = ['AAA', 'AA', 'A', 'BBB', 'AA', 'AAA', 'A', 'AAA']

# Contar ocurrencias
num_aaa = calificaciones.count('AAA')
num_aa = calificaciones.count('AA')
num_a = calificaciones.count('A')

print(f"Calificación AAA: {num_aaa} veces")  # 3
print(f"Calificación AA: {num_aa} veces")    # 2
print(f"Calificación A: {num_a} veces")      # 2
```

4.9 Método sort()

Ordena la lista in-place (modifica la lista original):

```
# Tasas de interés desordenadas
tasas = [0.065, 0.045, 0.075, 0.055, 0.050]
print(f"Tasas desordenadas: {tasas}")
```

```
# Ordenar ascendentemente
tasas.sort()
print(f"Tasas ordenadas (asc): {tasas}")
# [0.045, 0.05, 0.055, 0.065, 0.075]

# Ordenar descendentemente
tasas.sort(reverse=True)
print(f"Tasas ordenadas (desc): {tasas}")
# [0.075, 0.065, 0.055, 0.05, 0.045]

# Ordenar strings
paises = ['Perú', 'Argentina', 'Chile', 'Brasil', 'Colombia']
paises.sort()
print(f"Países ordenados: {paises}")
# ['Argentina', 'Brasil', 'Chile', 'Colombia', 'Perú']
```

4.10 Función sorted()

Retorna una nueva lista ordenada (no modifica la original):

```
# PIB de regiones
pib_regiones = [100000, 50000, 75000, 30000, 45000]
print(f"PIB original: {pib_regiones}")

# Crear lista ordenada sin modificar original
pib_ordenado = sorted(pib_regiones)
print(f"PIB ordenado: {pib_ordenado}")
print(f"PIB original (sin cambios): {pib_regiones}")

# Ordenar en reversa
pib_descendente = sorted(pib_regiones, reverse=True)
print(f"PIB descendente: {pib_descendente}")
```

4.11 Método reverse()

Invierte el orden de los elementos:

```
# Años en orden cronológico
anos = [2019, 2020, 2021, 2022, 2023]
print(f"Años cronológicos: {anos}")

# Invertir orden
anos.reverse()
print(f"Años invertidos: {anos}")
# [2023, 2022, 2021, 2020, 2019]

# Alternativa con slicing
anos_originales = [2019, 2020, 2021, 2022, 2023]
```

```
anos_invertidos = anos_originales[::-1]
print(f"Invertidos con slicing: {anos_invertidos}")
```

5 Diccionarios

Los diccionarios son colecciones de pares clave-valor, desordenadas (hasta Python 3.6) u ordenadas por inserción (Python 3.7+), mutables e indexables por claves. Son ideales para representar datos estructurados en economía.

5.1 Creación de diccionarios

```
# Diccionario vacío
pais_vacio = {}
print(type(pais_vacio))  # <class 'dict'>

# Diccionario con datos económicos de un país
peru = {
    'nombre': 'Perú',
    'pib': 242632,
    'poblacion': 33715471,
    'moneda': 'Sol',
    'codigo_iso': 'PER'
}
print(peru)

# Diccionario con diferentes tipos de valores
indicadores = {
    'pib': 242632,              # int
    'crecimiento': 2.7,          # float
    'moneda': 'PEN',             # string
    'emergente': True,           # boolean
    'sectores': ['Minería', 'Agricultura'],  # lista
    'comercio': {'exp': 63200, 'imp': 53800}  # diccionario anidado
}
print(indicadores)
```

5.2 Acceso a elementos

```
# Datos macroeconómicos
macro = {
    'pib': 242632,
    'inflacion': 3.5,
    'desempleo': 7.2,
    'tipo_cambio': 3.75
}

# Acceder usando corchetes
```

```
pib_actual = macro['pib']
print(f"PIB: {pib_actual}")

inflacion_actual = macro['inflacion']
print(f"Inflación: {inflacion_actual}%")

# Método get() (más seguro)
desempleo = macro.get('desempleo')
print(f"Desempleo: {desempleo}%")

# get() con valor por defecto
reservas = macro.get('reservas', 0)  # Retorna 0 si no existe
print(f"Reservas: {reservas}")

# Intentar acceder a clave inexistente con []
try:
    deuda = macro['deuda']
except KeyError:
    print("Error: 'deuda' no existe en el diccionario")
```

5.3 Modificación de diccionarios

```
# Indicadores económicos
indicadores = {
    'pib': 240000,
    'inflacion': 3.2,
    'desempleo': 7.5
}
print(f"Original: {indicadores}")

# Modificar valores existentes
indicadores['pib'] = 242632
indicadores['inflacion'] = 3.5
print(f"Actualizado: {indicadores}")

# Agregar nuevas claves
indicadores['tipo_cambio'] = 3.75
indicadores['reservas'] = 74500
print(f"Expandido: {indicadores}")

# Eliminar elementos
del indicadores['desempleo']
print(f"Después de eliminar: {indicadores}")
```

5.4 Verificación de claves

```
# Datos de un sector económico
sector = {
    'nombre': 'Minería',
    'contribucion_pib': 10.5,
    'empleo': 250000,
    'exportaciones': 30000
}

# Verificar si una clave existe
print('nombre' in sector)          # True
print('importaciones' in sector)   # False

# Verificar valores (menos eficiente)
print('Minería' in sector.values())  # True
print(10.5 in sector.values())       # True

# Uso práctico con condicionales
if 'contribucion_pib' in sector:
    print(f"Contribución al PIB: {sector['contribucion_pib']}%")
else:
    print("No hay datos de contribución al PIB")
```

6 Operaciones con diccionarios

6.1 Métodos keys(), values(), items()

```
# Exportaciones por sector (millones USD)
exportaciones = {
    'Minería': 30000,
    'Agricultura': 7500,
    'Pesca': 2800,
    'Manufactura': 12500
}

# Obtener todas las claves
sectores = exportaciones.keys()
print(f"Sectores: {list(sectores)}")
# ['Minería', 'Agricultura', 'Pesca', 'Manufactura']

# Obtener todos los valores
valores = exportaciones.values()
print(f"Valores: {list(valores)}")
# [30000, 7500, 2800, 12500]

# Calcular total de exportaciones
```

```
total_exportaciones = sum(exportaciones.values())
print(f"Total exportaciones: USD {total_exportaciones:,} millones")

# Obtener pares clave-valor
items = exportaciones.items()
print(f"Items: {list(items)}")
# [('Minería', 30000), ('Agricultura', 7500), ...]

# Iterar sobre items
print("\nDetalle de exportaciones:")
for sector, valor in exportaciones.items():
    porcentaje = (valor / total_exportaciones) * 100
    print(f"  {sector}: USD {valor:,} M ({porcentaje:.1f}%)")
```

6.2 Método update()

```
# Datos base
datos_2022 = {
    'pib': 240000,
    'inflacion': 3.0,
    'desempleo': 7.8
}
print(f"Datos 2022: {datos_2022}")

# Actualizar con nuevos datos
nuevos_datos = {
    'pib': 242632,
    'tipo_cambio': 3.75,
    'reservas': 74500
}

datos_2022.update(nuevos_datos)
print(f"Datos actualizados: {datos_2022}")
# {'pib': 242632, 'inflacion': 3.0, 'desempleo': 7.8,
#  'tipo_cambio': 3.75, 'reservas': 74500}
```

6.3 Método pop()

```
# Indicadores trimestrales
indicadores = {
    'Q1': 50000,
    'Q2': 52000,
    'Q3': 51500,
    'Q4': 53000
}
print(f"Original: {indicadores}")
```

```
# Eliminar y obtener un valor
valor_q1 = indicadores.pop('Q1')
print(f"Valor Q1 eliminado: {valor_q1}")
print(f"Indicadores restantes: {indicadores}")

# pop() con valor por defecto
valor_q5 = indicadores.pop('Q5', 0)
print(f"Valor Q5: {valor_q5}")  # 0 (no existe)
```

6.4 Método copy()

```
import copy

# Datos originales
datos_base = {
    'pib': 242632,
    'inflacion': 3.5,
    'sectores': ['Minería', 'Agricultura']
}

# Copia superficial
datos_copia = datos_base.copy()
datos_copia['pib'] = 250000
datos_copia['sectores'].append('Manufactura')

print(f"Base: {datos_base}")
print(f"Copia: {datos_copia}")
# Nota: 'sectores' se modifica en ambos

# Copia profunda para diccionarios anidados
datos_profunda = copy.deepcopy(datos_base)
datos_profunda['sectores'].append('Servicios')

print(f"\nBase después de deepcopy: {datos_base}")
print(f"Copia profunda: {datos_profunda}")
```

6.5 Método clear()

```
# Datos temporales
temp = {'a': 1, 'b': 2, 'c': 3}
print(f"Original: {temp}")

temp.clear()
print(f"Después de clear: {temp}")  # {}
```

6.6 Diccionarios anidados

```
# Datos económicos por país
america_latina = {
    'Peru': {
        'pib': 242632,
        'poblacion': 33715471,
        'moneda': 'PEN',
        'capital': 'Lima'
    },
    'Chile': {
        'pib': 301000,
        'poblacion': 19493184,
        'moneda': 'CLP',
        'capital': 'Santiago'
    },
    'Colombia': {
        'pib': 314000,
        'poblacion': 51516562,
        'moneda': 'COP',
        'capital': 'Bogotá'
    }
}

# Acceder a datos anidados
pib_peru = america_latina['Peru']['pib']
print(f"PIB de Perú: USD {pib_peru:,} millones")

capital_chile = america_latina['Chile']['capital']
print(f"Capital de Chile: {capital_chile}")

# Calcular PIB per cápita
for pais, datos in america_latina.items():
    pib_pc = (datos['pib'] / datos['poblacion']) * 1000000
    print(f"PIB per cápita {pais}: USD {pib_pc:,.2f}")
```

7 Tuplas

Las tuplas son colecciones ordenadas e inmutables. Son más eficientes en memoria que las listas y se usan cuando los datos no deben cambiar.

7.1 Creación de tuplas

```
# Tupla con paréntesis
coordenadas = (10.5, 15.3)
print(coordenadas)
print(type(coordenadas))  # <class 'tuple'>
```

```
# Tupla sin paréntesis (empaquetado de tupla)
punto = 5, 10
print(punto)
print(type(punto))  # <class 'tuple'>

# Tupla de un elemento (requiere coma)
tupla_un_elemento = (5,)
print(type(tupla_un_elemento))  # <class 'tuple'>

no_es_tupla = (5)  # Sin coma, es un int
print(type(no_es_tupla))  # <class 'int'>

# Tupla vacía
tupla_vacia = ()
print(tupla_vacia)

# Crear tupla desde lista
lista_tasas = [0.05, 0.06, 0.055, 0.065]
tupla_tasas = tuple(lista_tasas)
print(tupla_tasas)
```

7.2 Características de tuplas

```
# Datos macroeconómicos como tupla
macro_2023 = ('Peru', 242632, 3.5, 7.2)

# Indexación (igual que listas)
pais = macro_2023[0]
pib = macro_2023[1]
print(f"País: {pais}, PIB: {pib}")

# Slicing
indicadores = macro_2023[2:]
print(f"Indicadores: {indicadores}")  # (3.5, 7.2)

# Las tuplas son inmutables
try:
    macro_2023[1] = 250000
except TypeError:
    print("Error: Las tuplas no se pueden modificar")

# Pero pueden contener objetos mutables
tupla_con_lista = (1, 2, [3, 4])
tupla_con_lista[2].append(5)  # Modificar la lista interna
print(tupla_con_lista)  # (1, 2, [3, 4, 5])
```

7.3 Ventajas de tuplas

```
import sys

# Comparar uso de memoria
lista_grande = [i for i in range(1000)]
tupla_grande = tuple(i for i in range(1000))

memoria_lista = sys.getsizeof(lista_grande)
memoria_tupla = sys.getsizeof(tupla_grande)

print(f"Memoria lista: {memoria_lista} bytes")
print(f"Memoria tupla: {memoria_tupla} bytes")
print(f"Tupla usa {memoria_lista - memoria_tupla} bytes menos")

# Tuplas como claves de diccionarios
# (listas no pueden ser claves)
coordenadas_ciudades = {
    ('Lima', 'Peru'): (12.0464, 77.0428),
    ('Arequipa', 'Peru'): (16.4090, 71.5375),
    ('Cusco', 'Peru'): (13.5319, 71.9675)
}

lima_coords = coordenadas_ciudades[('Lima', 'Peru')]
print(f"Coordenadas de Lima: {lima_coords}")
```

7.4 Desempaquetado de tuplas

```
# Datos de una transacción
transaccion = ('Compra', 1000, 3.75, 'USD')

# Desempaquetado simple
tipo, cantidad, precio, moneda = transaccion
print(f"Tipo: {tipo}")
print(f"Cantidad: {cantidad}")
print(f"Precio: {precio}")
print(f"Moneda: {moneda}")

# Intercambiar valores sin variable temporal
a = 10
b = 20
print(f"Antes: a={a}, b={b}")

a, b = b, a  # Intercambio usando tuplas
print(f"Después: a={a}, b={b}")

# Desempaquetado extendido
```

```python
numeros = (1, 2, 3, 4, 5)
primero, *medio, ultimo = numeros
print(f"Primero: {primero}")    # 1
print(f"Medio: {medio}")        # [2, 3, 4]
print(f"Último: {ultimo}")      # 5
```

7.5 Métodos de tuplas

```python
# Tuplas tienen pocos métodos (porque son inmutables)
calificaciones = ('AAA', 'AA', 'A', 'BBB', 'AA', 'AAA', 'A', 'AAA')

# count() - contar ocurrencias
num_aaa = calificaciones.count('AAA')
print(f"Número de AAA: {num_aaa}")  # 3

# index() - encontrar índice
indice_bbb = calificaciones.index('BBB')
print(f"Índice de BBB: {indice_bbb}")  # 3

# Funciones estándar
print(f"Longitud: {len(calificaciones)}")  # 8

# Tuplas numéricas
precios = (100, 105, 98, 110, 95)
print(f"Máximo: {max(precios)}")   # 110
print(f"Mínimo: {min(precios)}")   # 95
print(f"Suma: {sum(precios)}")     # 508
```

8 Asignación múltiple con diccionarios

El desempaquetado funciona con diccionarios también:

```python
# Desempaquetar claves
datos = {'nombre': 'Peru', 'pib': 242632, 'moneda': 'PEN'}

# Obtener claves
nombre, pib, moneda = datos  # Solo las claves
print(f"Claves: {nombre}, {pib}, {moneda}")
# nombre, pib, moneda (strings de las claves)

# Obtener valores
nombre_v, pib_v, moneda_v = datos.values()
print(f"Valores: {nombre_v}, {pib_v}, {moneda_v}")
# Peru, 242632, PEN

# Desempaquetar items (mejor opción)
for clave, valor in datos.items():
    print(f"{clave}: {valor}")
```

8.1 Retorno múltiple de funciones

```
def calcular_estadisticas(valores):
    """Retorna múltiples estadísticas como tupla"""
    minimo = min(valores)
    maximo = max(valores)
    promedio = sum(valores) / len(valores)
    return minimo, maximo, promedio

# Datos de ventas
ventas = [45, 52, 48, 55, 60, 58, 62]

# Desempaquetar retorno
min_venta, max_venta, prom_venta = calcular_estadisticas(ventas)

print(f"Venta mínima: {min_venta}")
print(f"Venta máxima: {max_venta}")
print(f"Venta promedio: {prom_venta:.2f}")
```

9 Aplicaciones económicas

9.1 Ejemplo 1: Análisis de cartera de inversiones

```
# Cartera de inversiones
cartera = {
    'BBVA': {'cantidad': 100, 'precio_compra': 25.50, 'precio_actual': 27.80},
    'BCP': {'cantidad': 150, 'precio_compra': 180.00, 'precio_actual': 195.50},
    'Credicorp': {'cantidad': 50, 'precio_compra': 150.00, 'precio_actual': 148.00}
}

print("ANÁLISIS DE CARTERA DE INVERSIONES")
print("=" * 60)

inversion_total = 0
valor_actual_total = 0
ganancia_total = 0

for accion, datos in cartera.items():
    cantidad = datos['cantidad']
    precio_compra = datos['precio_compra']
    precio_actual = datos['precio_actual']

    inversion = cantidad * precio_compra
    valor_actual = cantidad * precio_actual
    ganancia = valor_actual - inversion
```

```
    rentabilidad = (ganancia / inversion) * 100

    inversion_total += inversion
    valor_actual_total += valor_actual
    ganancia_total += ganancia

    print(f"\n{accion}:")
    print(f"  Cantidad: {cantidad} acciones")
    print(f"  Precio compra: S/ {precio_compra:.2f}")
    print(f"  Precio actual: S/ {precio_actual:.2f}")
    print(f"  Inversión: S/ {inversion:,.2f}")
    print(f"  Valor actual: S/ {valor_actual:,.2f}")
    print(f"  Ganancia/Pérdida: S/ {ganancia:,.2f}")
    print(f"  Rentabilidad: {rentabilidad:+.2f}%")

rentabilidad_total = (ganancia_total / inversion_total) * 100

print("\n" + "=" * 60)
print("RESUMEN TOTAL:")
print(f"  Inversión total: S/ {inversion_total:,.2f}")
print(f"  Valor actual total: S/ {valor_actual_total:,.2f}")
print(f"  Ganancia/Pérdida total: S/ {ganancia_total:,.2f}")
print(f"  Rentabilidad total: {rentabilidad_total:+.2f}%")
```

9.2 Ejemplo 2: Análisis de comercio exterior

```
# Datos de comercio por país (millones USD)
comercio_exterior = {
    'China': {'exportaciones': 15000, 'importaciones': 12000},
    'USA': {'exportaciones': 8500, 'importaciones': 10500},
    'Brasil': {'exportaciones': 2500, 'importaciones': 3000},
    'Chile': {'exportaciones': 1800, 'importaciones': 2200}
}

print("ANÁLISIS DE COMERCIO EXTERIOR")
print("=" * 60)

total_exp = 0
total_imp = 0

resultados = []

for pais, datos in comercio_exterior.items():
    exp = datos['exportaciones']
    imp = datos['importaciones']
    balanza = exp - imp
    comercio_total = exp + imp
```

```
    total_exp += exp
    total_imp += imp

    resultados.append((pais, exp, imp, balanza, comercio_total))

# Ordenar por volumen de comercio total
resultados.sort(key=lambda x: x[4], reverse=True)

# Mostrar resultados
for pais, exp, imp, balanza, comercio in resultados:
    print(f"\n{pais}:")
    print(f"  Exportaciones: USD {exp:,} M")
    print(f"  Importaciones: USD {imp:,} M")
    print(f"  Balanza comercial: USD {balanza:+,} M")
    print(f"  Comercio total: USD {comercio:,} M")

    if balanza > 0:
        print(f"  Estado: Superávit")
    elif balanza < 0:
        print(f"  Estado: Déficit")
    else:
        print(f"  Estado: Equilibrio")

balanza_total = total_exp - total_imp

print("\n" + "=" * 60)
print("TOTALES:")
print(f"  Exportaciones totales: USD {total_exp:,} M")
print(f"  Importaciones totales: USD {total_imp:,} M")
print(f"  Balanza comercial total: USD {balanza_total:+,} M")
```

9.3 Ejemplo 3: Sistema de calificación de riesgo

```
# Empresas y sus indicadores financieros
empresas = [
    ('TechCorp', {'liquidez': 1.8, 'endeudamiento': 0.45, 'rentabilidad': 0.15}),
    ('IndusPeru', {'liquidez': 1.2, 'endeudamiento': 0.65, 'rentabilidad': 0.08}),
    ('AgroSur', {'liquidez': 2.5, 'endeudamiento': 0.30, 'rentabilidad': 0.12}),
    ('MineroX', {'liquidez': 1.5, 'endeudamiento': 0.55, 'rentabilidad': 0.18})
]

def calificar_empresa(indicadores):
    """Califica una empresa según sus indicadores"""
    liquidez = indicadores['liquidez']
    endeudamiento = indicadores['endeudamiento']
    rentabilidad = indicadores['rentabilidad']
```

```
    puntos = 0

    # Evaluar liquidez
    if liquidez >= 2.0:
        puntos += 3
    elif liquidez >= 1.5:
        puntos += 2
    elif liquidez >= 1.0:
        puntos += 1

    # Evaluar endeudamiento (menor es mejor)
    if endeudamiento <= 0.3:
        puntos += 3
    elif endeudamiento <= 0.5:
        puntos += 2
    elif endeudamiento <= 0.7:
        puntos += 1

    # Evaluar rentabilidad
    if rentabilidad >= 0.15:
        puntos += 3
    elif rentabilidad >= 0.10:
        puntos += 2
    elif rentabilidad >= 0.05:
        puntos += 1

    # Determinar calificación
    if puntos >= 8:
        return 'AAA', puntos
    elif puntos >= 6:
        return 'AA', puntos
    elif puntos >= 4:
        return 'A', puntos
    elif puntos >= 2:
        return 'BBB', puntos
    else:
        return 'BB', puntos

print("SISTEMA DE CALIFICACIÓN DE RIESGO")
print("=" * 70)

calificaciones = []

for empresa, indicadores in empresas:
    calificacion, puntos = calificar_empresa(indicadores)
    calificaciones.append((empresa, calificacion, puntos, indicadores))
```

```
    print(f"\n{empresa}:")
    print(f"  Liquidez: {indicadores['liquidez']:.2f}")
    print(f"  Endeudamiento: {indicadores['endeudamiento']:.2%}")
    print(f"  Rentabilidad: {indicadores['rentabilidad']:.2%}")
    print(f"  Puntos: {puntos}/9")
    print(f"  Calificación: {calificacion}")

# Ordenar por calificación
calificaciones.sort(key=lambda x: x[2], reverse=True)

print("\n" + "=" * 70)
print("RANKING DE EMPRESAS:")
for i, (empresa, calif, puntos, _) in enumerate(calificaciones, 1):
    print(f"{i}. {empresa}: {calif} ({puntos} puntos)")
```

10 Ejercicios prácticos

10.1 Ejercicio 1: Administrador de presupuesto familiar

```
# Sistema de presupuesto mensual
presupuesto = {
    'Alimentos': 1200,
    'Transporte': 300,
    'Vivienda': 800,
    'Servicios': 250,
    'Educación': 400,
    'Salud': 200,
    'Entretenimiento': 150
}

gastos_reales = {
    'Alimentos': 1250,
    'Transporte': 280,
    'Vivienda': 800,
    'Servicios': 270,
    'Educación': 400,
    'Salud': 180,
    'Entretenimiento': 200
}

print("ANÁLISIS DE PRESUPUESTO FAMILIAR")
print("=" * 70)

presupuesto_total = sum(presupuesto.values())
gastos_totales = sum(gastos_reales.values())

print(f"\nPresupuesto total: S/ {presupuesto_total:,.2f}")
```

```
print(f"Gastos totales: S/ {gastos_totales:,.2f}")
print(f"Diferencia: S/ {presupuesto_total - gastos_totales:+,.2f}")

print("\nDETALLE POR CATEGORÍA:")
print("-" * 70)

categorias_excedidas = []
categorias_dentro = []
ahorros_por_categoria = []

for categoria in presupuesto.keys():
    presu = presupuesto[categoria]
    gasto = gastos_reales[categoria]
    diferencia = presu - gasto
    porcentaje = (gasto / presu) * 100

    print(f"\n{categoria}:")
    print(f"  Presupuestado: S/ {presu:,.2f}")
    print(f"  Gastado: S/ {gasto:,.2f}")
    print(f"  Diferencia: S/ {diferencia:+,.2f}")
    print(f"  Ejecución: {porcentaje:.1f}%")

    if diferencia < 0:
        print(f"  Estado:   EXCEDIDO")
        categorias_excedidas.append((categoria, abs(diferencia)))
    else:
        print(f"  Estado:   Dentro del presupuesto")
        categorias_dentro.append((categoria, diferencia))
        ahorros_por_categoria.append(diferencia)

print("\n" + "=" * 70)
print("RESUMEN:")

if categorias_excedidas:
    print("\nCategorías excedidas:")
    for cat, exceso in categorias_excedidas:
        print(f"  - {cat}: S/ {exceso:,.2f} de exceso")

if categorias_dentro:
    print("\nCategorías con ahorro:")
    for cat, ahorro in categorias_dentro:
        print(f"  - {cat}: S/ {ahorro:,.2f} de ahorro")

if gastos_totales > presupuesto_total:
    print(f"\n  Déficit total: S/ {abs(presupuesto_total - gastos_totales):,.2f}")
else:
    print(f"\n  Ahorro total: S/ {presupuesto_total - gastos_totales:,.2f}")
```

10.2 Ejercicio 2: Análisis de series de tiempo económicas

```
# PIB trimestral de múltiples años
pib_trimestral = {
    2021: [48000, 49500, 50000, 51000],
    2022: [51500, 52500, 53000, 54000],
    2023: [54500, 55500, 56000, 57000]
}

print("ANÁLISIS DE SERIES DE TIEMPO - PIB TRIMESTRAL")
print("=" * 70)

# Calcular PIB anual
for ano, trimestres in pib_trimestral.items():
    pib_anual = sum(trimestres)
    print(f"\nAño {ano}:")
    print(f"  PIB anual: S/ {pib_anual:,} millones")

    # PIB por trimestre
    for i, pib in enumerate(trimestres, 1):
        porcentaje = (pib / pib_anual) * 100
        print(f"  Q{i}: S/ {pib:,} M ({porcentaje:.1f}%)")

    # Crecimiento intertrimestral
    print(f"\n  Crecimiento intertrimestral:")
    for i in range(1, len(trimestres)):
        crecimiento = ((trimestres[i] - trimestres[i-1]) / trimestres[i-1]) * 100
        print(f"    Q{i} a Q{i+1}: {crecimiento:+.2f}%")

# Análisis interanual
print("\n" + "=" * 70)
print("CRECIMIENTO INTERANUAL:")

anos = sorted(pib_trimestral.keys())
for i in range(1, len(anos)):
    ano_anterior = anos[i-1]
    ano_actual = anos[i]

    pib_ant = sum(pib_trimestral[ano_anterior])
    pib_act = sum(pib_trimestral[ano_actual])

    crecimiento = ((pib_act - pib_ant) / pib_ant) * 100

    print(f"{ano_anterior} a {ano_actual}: {crecimiento:+.2f}%")

# Identificar mejor y peor trimestre
todos_trimestres = []
for ano, trimestres in pib_trimestral.items():
```

```
    for i, pib in enumerate(trimestres, 1):
        todos_trimestres.append((ano, i, pib))

mejor_trimestre = max(todos_trimestres, key=lambda x: x[2])
peor_trimestre = min(todos_trimestres, key=lambda x: x[2])

print("\n" + "=" * 70)
print(f"Mejor trimestre: Q{mejor_trimestre[1]} {mejor_trimestre[0]} - S/ {mejor_trime
print(f"Peor trimestre: Q{peor_trimestre[1]} {peor_trimestre[0]} - S/ {peor_trimestre
```

10.3 Ejercicio 3: Sistema de inventario

```
# Inventario de productos
inventario = {
    'PROD001': {'nombre': 'Laptop', 'cantidad': 25, 'precio': 2500, 'minimo': 10},
    'PROD002': {'nombre': 'Mouse', 'cantidad': 100, 'precio': 25, 'minimo': 50},
    'PROD003': {'nombre': 'Teclado', 'cantidad': 8, 'precio': 80, 'minimo': 15},
    'PROD004': {'nombre': 'Monitor', 'cantidad': 30, 'precio': 800, 'minimo': 20},
    'PROD005': {'nombre': 'Impresora', 'cantidad': 5, 'precio': 450, 'minimo': 8}
}

print("SISTEMA DE GESTIÓN DE INVENTARIO")
print("=" * 70)

valor_total_inventario = 0
productos_bajo_stock = []
productos_ok = []

for codigo, producto in inventario.items():
    nombre = producto['nombre']
    cantidad = producto['cantidad']
    precio = producto['precio']
    minimo = producto['minimo']

    valor_producto = cantidad * precio
    valor_total_inventario += valor_producto

    estado = "  Stock OK" if cantidad >= minimo else "  BAJO STOCK"

    print(f"\nCódigo: {codigo}")
    print(f"  Producto: {nombre}")
    print(f"  Cantidad: {cantidad} unidades")
    print(f"  Precio unitario: S/ {precio:.2f}")
    print(f"  Valor en inventario: S/ {valor_producto:,.2f}")
    print(f"  Stock mínimo: {minimo} unidades")
    print(f"  Estado: {estado}")
```

```
    if cantidad < minimo:
        faltante = minimo - cantidad
        productos_bajo_stock.append((nombre, cantidad, minimo, faltante))
    else:
        productos_ok.append((nombre, cantidad))

print("\n" + "=" * 70)
print("RESUMEN DE INVENTARIO:")
print(f"Valor total del inventario: S/ {valor_total_inventario:,.2f}")
print(f"Productos en stock adecuado: {len(productos_ok)}")
print(f"Productos bajo stock mínimo: {len(productos_bajo_stock)}")

if productos_bajo_stock:
    print("\n  ALERTA - Productos que requieren reposición:")
    for nombre, actual, minimo, faltante in productos_bajo_stock:
        print(f"  - {nombre}: {actual} unidades (faltante: {faltante} unidades)")

print("\n" + "=" * 70)
print("PRODUCTOS POR VALOR:")
productos_valor = [(codigo, prod['nombre'], prod['cantidad'] * prod['precio'])
                   for codigo, prod in inventario.items()]
productos_valor.sort(key=lambda x: x[2], reverse=True)

for i, (codigo, nombre, valor) in enumerate(productos_valor, 1):
    porcentaje = (valor / valor_total_inventario) * 100
    print(f"{i}. {nombre} ({codigo}): S/ {valor:,.2f} ({porcentaje:.1f}%)")
```

11 Conclusión

En esta guía hemos explorado las estructuras de datos fundamentales de Python:

Listas

Colecciones ordenadas y mutables. Ideales para secuencias de datos que pueden cambiar. Métodos importantes: append(), extend(), insert(), remove(), pop(), sort().

Diccionarios

Colecciones de pares clave-valor. Perfectos para datos estructurados y búsquedas rápidas. Métodos importantes: keys(), values(), items(), get(), update(), pop().

Tuplas

Colecciones ordenadas e inmutables. Más eficientes en memoria que las listas. Útiles para datos que no deben cambiar y como claves de diccionarios.

Conceptos importantes

La diferencia entre asignación por referencia y por valor es crucial. Usar copy.copy() para copias superficiales y copy.deepcopy() para copias profundas de estructuras anidadas.

12 Próximos pasos

En la siguiente guía (Guía 4: Ejecución Condicional) exploraremos:

- Expresiones booleanas
- Operadores lógicos

- Ejecuciones condicionales (if, elif, else)
- Condicionales anidados
- Manejo de excepciones con try-except
- Evaluación de cortocircuito

Estos conceptos nos permitirán crear programas que tomen decisiones basadas en condiciones, esenciales para análisis económico automatizado.

13 Recursos adicionales

Para profundizar en estructuras de datos:

- Python Data Structures: docs.python.org/tutorial/datastructures.html
- Práctica con ejercicios económicos reales
- Exploración de bibliotecas como Pandas para análisis avanzado

La práctica constante con datos económicos reales consolidará estos conceptos. Se recomienda trabajar con datos de fuentes como BCRP, INEI, o Banco Mundial para aplicar estas estructuras en contextos realistas.

14 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!